RokidLab
Rokid AR 眼镜的手机端 AI 智能体平台

项目背景 · 技术架构 · 应用场景 · 创新点

一个手机 App，同时承担三件事：
把眼镜的能力边界推到系统之外，
把 AI 从「会聊天」推到「会动手」，
并让造 AIUI 应用这件事不再需要写代码。

版本	v4.4（手机端 versionCode 29）
配套端	RokidLink v4.4（眼镜端 versionCode 22）
技术栈	Kotlin · Jetpack Compose · CXR-L SDK · 自研 ADB / HID 协议栈
平台	Android 10+（手机端 minSdk 29 / 眼镜端 minSdk 28）
开源许可	MIT License · 作者 DLOVER

目录

CONTENTS

01	**项目背景**	眼镜的封闭边界，与三个绕不开的现实约束
02	**技术架构**	六层依赖、双端结构、四条通信链路、Agent 内核
03	**应用场景**	从日常助理到无代码开发，从个人使用到生态共建
04	**创新点**	六项可被验证的架构与工程突破
05	**工程数据**	规模、质量门禁与版本演进

本文所有数据均取自项目仓库实测：工具与 Provider 数量来自构建门禁脚本 check_tool_wiring.py 的运行结果，测试用例数为 @Test 注解总数，语言条目数为 strings.xml 实际条目统计，版本号取自 Gradle 配置。

项目背景

眼镜的封闭边界，与三个绕不开的现实约束

01

AR 眼镜的想象空间很大，但真正落到一台设备上时，开发者面对的是一片被系统层、硬件协议和供应链三重封住的区域。RokidLab 的起点，不是「做一个 AI 助手」，而是先把这段被封住的路径一寸寸走通。

三个现实约束

— **约束一 • 设备无法直连 PC。**眼镜侧不提供标准 adb 接入，传统「连电脑 → push APK → 调试」的链路在移动场景下完全不可用，文件传输、应用安装、系统信息读取全部需要在手机上自己从零实现。

— **约束二 • 硬件接口非标准。**眼镜端内核的 HID 驱动不支持标准 Gamepad Input Device，所有按键必须经 Keyboard 与 Consumer Control 两个通道转译，触控与文字输入都需要单独设计降级链路。

— **约束三 • 屏幕是单色小尺寸。**480×640

单绿色波导屏，深蓝灰在此屏幕上几乎等同于黑色，这意味着常规移动端 UI 迁移过来会大面积「看不见」。

由此拆出的四个建设目标

01

把协议层补齐

自研 ADB over TCP 完整协议栈与蓝牙 HID 设备端，让「不依赖 adb.exe、不依赖 Termux、不依赖 PC」成为可能。

02

把 AI 变成执行体

不止于对话，而是给它 105 个真实工具、长期记忆、知识库与本机执行环境，让它能查、能改、能跑。

03

把开发门槛降到零

用对话直接生成 AIUI 智能体应用并在眼镜上运行，让非开发者也能做眼镜端小应用。

04

把生态开放出去

内置应用商店与开发者提交流程，第三方可自行上架，形成可持续的应用供给。

一句话定位

RokidLab 是 Rokid 眼镜的官方开源配套应用，也是一个跑在手机上的 AI 智能体运行时：它一边替眼镜补齐缺失的系统能力（文件、安装、调试、投屏、输入），一边把云端大模型接到眼镜的物理世界（相机、屏幕、麦克风、六轴 IMU），最终让「说一句话」变成「一件事被做完」。

02

技术架构

六层依赖 · 双端结构 · 四条通信链路 · Agent 内核

双端结构：手机端算力与眼镜端执行的分工 2.1

phone-app 提供全部智能与算力，RokidLink 提供触达眼镜的通道

模块	职责
phone-app（手机端）	全部智能与算力在此：Agent 主循环、105 个工具、模型客户端、记忆与知识库、SQLite、proot 容器、UI。minSdk 29，Compose + Material 3。
RokidLink（眼镜端）	轻量常驻宿主：AIUI 渲染宿主、ASR 推送服务、蓝牙隧道、投屏接收、按键桥、文字输入服务。minSdk 28，R8 混淆后约 8.7 MB。

眼镜端 APK 由 Gradle 自动同步进手机端 assets，二者协议常量（LinkProtocol / AiChannel）双端逐字节同源，由构建期门禁 checkProtocolSynced 守护。**所有 AI 能力都在手机上，模型可任意替换，眼镜侧无需任何改动。**

手机端六层架构：依赖单向，反向依赖即违规 2.2

每层只依赖下一层，platform/ 是全仓唯一允许出现反射的包

L5 — **app • feature • store • settings**
UI 与入口 + 手动 DI 容器 AppContainer

L4 — **ai/**
Agent：工具注册 · 审批闸门 · 模型接缝 · 上下文压缩

L3 — **domain/**
领域服务：连接 · 授权 · 设备 · AI 会话 · AIUI · 投屏 · 文件 · 拍照

L2 — **glasses/**
会话层：CXR-L 会话 · LinkProtocol · ASR 桥 · 能力握手

L1 — **connection/ • adb/**
通道仲裁 · 双线路路由 · 自研 ADB 协议栈

L0 — **platform/**
平台适配：SDK 反射 · HID 桥 · ADB 传输 · ROM 适配（唯一反射边界）

这一约束带来的直接收益：SDK 或 ROM 升级时只需要改 **L0** 一个包；UI 层不可能误 import 传输层或反射层；跨 feature 的长生命周期对象由 **AppContainer** 一个手动 DI 容器装配，因此项目不需要引入 Hilt。

四条通信链路与「同设备单通道」仲裁 2.3

连接不可靠是常态，因此每条链路都配了降级与自愈

链路	协议 / 承载	承载能力	降级策略
CXR-L	Rokid 官方 SDK	安装 / 启动 / 语音播报 / 系统指令	能力握手三态，旧版按 v1 快速降级
ADB over TCP	自研协议栈（24B 小端报头 + RSA 认证）	文件管理 / 应用管理 / Shell / 屏幕镜像	WiFi 直连优先，失败回退蓝牙隧道
蓝牙 HID	BluetoothHidDevice 三通道描述符	手机变键盘 / 鼠标 / 手柄	Ctrl+V → ADB 粘贴 → TCP 剪贴板 三层降级
蓝牙 RFCOMM 隧道	自定义转发（握手令牌 ROKIDPUSH1）	ASR 文字回传 / AIUI 工具调用上行	同设备同 SCN 仅允许一条，ChannelArbiter 按优先级仲裁

通道仲裁的价值：手机蓝牙栈对同一设备同一 SCN 只允许一条客户端通道。项目把它收敛为「按优先级持有租约」——BACKGROUND（ASR 兜底补读）/

NORMAL（控制面）/
LONG_LIVED（投屏、镜像、文件浏览）。高优先级占用时低优先级主动让路，LONG_LIVED 持有超 10 分钟打疑似泄漏告警。这解决的是一类很具体的线上故障：某处漏掉 release 导致通道被永久占死。

Agent 内核：工具、审批、记忆、知识库 2.4

105 个工具 / 21 个 Provider / 17 个域，由构建门禁保证三表一致

（1）工具体系

工具按域拆分为 21 个 **ToolProvider**，执行分发与声明分离。新增一个工具只需在对应 Provider 的 tools() 加一条声明、在 execute() 加一个分支，风险档、schema、副作用、要眼镜、过程文案全部从这一条声明派生。风险档写在使用它的结构体里，**漏登记在结构上已不可能**；完全未知的名字（模型幻觉）按最保守档处理并被单调拒绝。

（2）审批闸门：唯一入口 + 八道 guard

每一次工具调用（无论来自对话还是 AIUI 页面）都先过 ApprovalGate.preExecute()，由 8 个策略源按固定顺序合成：页面准入 → 未知工具 → 工具开关 → 无人值守范围 → 来源限流 → 页面提权域 → 眼镜依赖 → 风险确认。

语义	定义
Deny 短路	任一 guard 拒绝即最终拒绝，不给后续 guard 翻案机会
首个 Ask 胜出	多个 guard 都想确认时，取第一个，避免用户被反复追问
null ≠ 表态	guard 返回 null 表示不表态；显式 Allow 会终止求值
fail-closed	AIUI 页面调 shell / screen 域时问不到用户即拒绝
工具自声明	问不到用户时放行还是拒绝，由该工具的 confirmPolicy 决定

确认通道有两条：**眼镜端优先、手机端兜底**。眼镜在线时行为与改造前完全一致；眼镜不在线（「本机模式」）时由手机弹窗承接，补上「本机模式要求不连眼镜」与「确认通道要求眼镜在线」在定义上互斥这个洞。通道不可用时如实返回 false，**绝不假装问过**。

（3）记忆三层与知识库

层	存储与策略	解决的问题
会话记忆	agent_sessions/<id>.jsonl 事件流；超长时前缀折叠为 ≤800 字 pinned system message	跨数小时长会话不失忆；模型可见内容全程可回放
长期记忆	SQLite + mem0 式评分淘汰（相关度/新鲜度/重要度）+ 词法 2-gram 与向量余弦 RRF 混合检索（k=60）	记住称呼与偏好，但不随记忆库增长撑爆 system prompt
知识库	txt 导入经编码链判定（BOM → UTF-16 → UTF-8 → GB18030）；BM25 IDF + 向量语义双路召回	中文 txt 乱码根治；回答可引用 docName/chunkIdx 来源

（4）上下文成本：从结构上省，而不是从描述上省

全量 105 个工具的 Schema 每轮约 13~14k token，是长会话最大成本项。项目用两刀结构性砍掉它：**闲聊快路**对高置信度纯闲聊轮装配零工具、换用精简人设；**工具意图路由**对任务轮按名白名单装配，从全量降到个位数。路由子集不会自动扩全量——多步任务靠并集，清单外的能力靠显式逃生舱，另有一轮约 2k 的轻量菜单探测供模型点名。

这里有一条重要的工程取舍：**覆盖率不是目标，误放率才是**。误放的后果是模型在残缺工具集里自信作答，比漏放更难发现。因此高危长尾（发短信、卸载、装包）故意保留全量，走显式确认。

执行层与感知层：两个不借外力的自持方案

2.5

把「让 AI 动手」这件事从依赖第三方 App 变成 App 自身能力

本机执行层（proot 自持）

App 内自持 proot 容器跑完整 Ubuntu。随包仅 214 KB 的两个 so，rootfs 与 Python / Node.js 等组件全部运行期按需下载。AI 可直接执行 shell、跑 Python、apt 装包，并在容器里拉起常驻网页服务供手机实时预览。不依赖 Termux，也不需要用户额外安装任何终端应用。

屏幕操作域（无障碍自持）

自建无障碍服务提供零弹窗截屏、读屏、代点击与输入。读屏取元素树而非截图，token 消耗降一个数量级且定位更准；点击按文案或编号回树重查，界面变化时如实告知而不是照旧坐标盲点。刻意不代填密码；提交类与裸坐标点击需用户确认。

感知输入扩展

眼镜端 20Hz 上报六轴 IMU，判定全部在手机端完成，支持点头 / 摇头 / 静止 / 持续抬头等规则，动作可放开为任意工具调用，可用语音直接建规则，审批弹窗也支持点头确认、摇头取消。

本地模型与本地 OCR

内置 Ollama 连接器，可在无云端 API 的情况下跑本地模型对话；OCR 采用 PP-OCRv4 + ONNX Runtime 完全离线推理，不依赖网络与 GMS，兼容 16KB 页面设备。

03

应用场景

从日常助理到无代码开发，从个人使用到生态共建

眼镜上的日常助理

3.1

眼镜不适手，手机是天然的第二块屏幕

查眼镜状态
电量、存储、已装应用、当前时间，一句话问完。整眼镜信息已收敛为单一工具 get_glasses_status，避免多工具拼装。

看时间定提醒
「5 分钟后提醒我喝水」「明早 8 点叫我」「17 点打开小智」——到点由眼镜语音播报，可同时启动指定应用，App 退到后台也照样触发。

放一首歌
语音点歌，音频先落本地再播放，规避 CDN 返回 octet-stream 导致的解码器选不出问题；专辑封面在手机侧转 data URL 下发，眼镜本身无网络也能显示。

查天气与算账
天气走 Open-Meteo 免 API Key；计算由内置递归下降表达式解析器完成，纯 Kotlin 无依赖，不依赖模型算错。

操控手机
电话拨打、联系人搜索、日历读写、设闹钟、调音量、打开手机应用、手电筒、导航、查步数——手机域 12 个工具统一受同一套审批闸门管辖。

看看眼前
把眼镜相机暴露成模型可主动调用的工具。模型支持看图则走多模态，不支持则兜底本地 OCR 转文字，两种情况下都不会编造「没有相机权限」。

拍照问 AI 与本地知识库：离线可用的学习闭环

3.2

整个链路不依赖眼镜联网，也不依赖任何云端服务

按一下镜腿按键，眼镜拍照 → 手机本地 OCR 识别题目文字 → 知识库检索（RAG）→ 大模型生成答案 → 眼镜语音播报。这条链路的意义在于：**眼镜无网络时，学习类问答依然完整可用。**

眼镜相机 → 本地 OCR → 知识库 RAG → 大模型 → 眼镜播报

知识库侧的关键细节是**编码判定**：中文用户的 txt 常是 GBK 或 UTF-16，按 UTF-8 硬解会整篇变成替换字符，而文件名与字节数照旧正确——症状是「导入看着成功、里面的内容永远问不出来」。项目建立 BOM → 无 BOM UTF-16 → 严格 UTF-8 试解 → GB18030
兜底的判定链，并在管理弹窗里直接显示块数与编码、可预览正文，让「到底导进去了什么」可见。

对话即开发：AIUI 智能体生成

3.3

说「做个番茄钟」，几秒后一个应用跑在眼镜上

这是项目最激进的一条产品线。用户与乐奇对话，AI 自动生成完整 AIUI 项目（.ink 页面 + app.json 清单），打包为 .aix 推送到眼镜渲染运行——**无需写代码，无需打开 IDE。**

— **技能体系内置**：官方组件规范、API 参考与渲染设计指南作为 Skill 文档内置在 assets，首次命中需求即自动加载，按需按章节精读，避免全量注入浪费 token。

— **先读再改**：对已生成的应用不满意时，直接说「把番茄钟改成红色」「加个暂停按钮」，AI 先读取项目现有源码，基于真实代码精准修改再覆盖写回，不凭记忆整文件重编。

— **启动参数直达**：说「用 AIUI 播放西厢」，启动参数随包下发，页面 boot 完成后作为首条 launch 消息投递；同一个包再次打开换参数时不重启宿主，直接更新参数下发，避免打断当前播放。

— **内容指纹版本控制**：VERSION 字段用内容指纹，代码一变眼镜端自动重新解压渲染，不需要手动清缓存。

Lab 工具桥：让眼镜上的页面能操作真实世界

3.4

把 AIUI 从「能渲染的静态页」变成「有能力的微前端」

AIUI 页面跑在眼镜端 ink 沙箱（quickjs-wasm）里，与 WebView 主 realm 隔离，原本只能渲染。Lab 工具桥让它可以调用手机端的真实能力：

```
await globalThis.Lab.callTool('play_song', { songName: '西厢' })
```

协议链路是一条完整的往返：页面 fetch 拦截 → 眼镜端宿主同步阻塞取结果 → 复用既有 RFCOMM 通道上行 → 手机端协调器分流 → 工具网关执行 → 结果按原路回传兑现 Promise。**不新开通道**是刻意的选择，因为眼镜端同一时刻只允许一条 RFCOMM 隧道。

— **能力策略是全域开放 + 有限黑名单。**刻意不按「危险」逐工具封禁——判据是「能力形态」与「一次调用是否触发多次模型调用且不可取消」，而不是「这个工具危不危险」。命令执行与整机触摸注入这两个原语，对第三方页面改为**知情同意**（fail-closed 确认），而不是整域摘除。

— **摘域的代价被认真评估过：**整域封禁会让 AIUI 页面路径整体断链，眼镜上「能说不能做」。因此选择让高风险原语走确认，而不是让整条路断掉。

— **典型成品：**音乐播放器 AIUI 应用，专辑封面全屏铺满、底部三行滚动歌词，封面与歌词物理分区绝不重叠，界面不保留任何播放控件——语音点歌与快捷键控制仍在，但界面只呈现内容。

眼镜的输入输出外设：手柄、投屏、文件与调试 3.5

把眼镜当作一台正经的设备来用，而不只是显示器

蓝牙手柄
手机模拟键盘 / 鼠标 / 手柄控制眼镜光标与游戏。方向键走 Consumer Control，功能键走 Keyboard，双通道并行。14 个按键位置可拖拽自定义，鼠标模式下方提供键盘按钮直接打字。

双向投屏
眼镜屏幕实时显示到手机（自研 ADB 隧道 + scrcpy-server，H.264 硬编零拷贝解码）；手机画面投射到眼镜（MediaProjection + Socket）。已适配华为 / 小米 / OPPO / vivo 投屏黑屏并自动降级。

文件管理
通过自研 ADB 协议浏览、上传、下载、重命名、复制、删除眼镜文件，支持图片预览与文本查看，可直接安装本地 APK。中文目录与中文文件名的超时问题已修复。

ADB 工具箱
不依赖电脑 adb.exe，直接在手机上完成系统信息查看、应用启停卸载冻结、Shell 命令、定时推送通知与定时启动应用。四个功能按钮用不同导航栏色区分，降低误触。

定时任务
设定间隔与次数，通过 ADB 向眼镜推送通知并触发语音播报，App 退后台也能到点执行，持久化 + 常驻触发。

应用商店
浏览、搜索、安装、更新，多商店源切换，22 种分类标签中英文本地化，支持下拉刷新与精选应用横向滑动。开发者可在设置页填表单一键提交到商店注册表。

主动式陪伴与环境聆听 3.6

从「你问我答」走向「它在合适的时候开口」

AI 不再只被动应答。5 种陪伴场景（勿扰 / 日程助手 / 轻陪伴 / 朋友 / 全陪）× 4 档闲聊频率，用户能看到「今天还可发 N 条」的额度。7 项主动能力各有各的时机，且不占闲聊额度：早晚安按作息、空闲闲聊、主动回访到点追问聊过的约定、日程简报提前 2 小时、行走中定时看看、久坐久走低头关怀、会话内沉默追击。

— **决策门统一准入：**静默冷却 → 主动收敛期 → 免打扰时段 → 每日额度与间隔。说一句「别烦我」即全局静默 24 小时。

— **响应率自适应调频：**10 分钟内回复记为「有回应」，间隔随回应率自动收缩或放宽；连续被忽略则进入收敛期。

— **时机内容绑定：**开口前先查天气与时间，没有内容就静默收回，不硬开口。同一时刻只跑一路主动轮，避免问候与关怀互相打断、白耗额度。

— **环境聆听**：眼镜端持续听周围，两级 VAD 端点检测（预判 + 确认），回答期真闭麦、播完自动重开——支持不摘眼镜的连续对话。

04

创新点

六项可被验证的架构与工程突破

六项创新点

4.1

每一条都对应一个可复现的验证方式，而非概念描述

01

对话即开发：AIUI 应用的零代码生产链路

把「造一个眼镜端应用」这件事从 IDE 里解放出来，变成一次对话。生成 → 推送 → 渲染 → 语音改 → 重装，全链路在对话中闭环。配套的 Skill 体系让模型自己学会官方组件规范，「先读再改」的修改范式避免了生成式编程最常见的整文件重编覆写。

02

自研 ADB 协议栈 + 三层输入降级：把 PC 从链路里彻底删掉

从零实现 ADB over TCP 完整协议（24 字节小端报头、RSA 认证握手、sync 上传下载、CLSE 正确消费），不依赖 adb.exe。文字输入设计为 TCP 剪贴板 → ADB 原始协议粘贴 → HID Ctrl+V 三层依次降级，任何一层可用都能送达。

03

Lab 工具桥：以「能力形态」而非「危险程度」划定页面能力边界

AIUI 页面能调用手机端全域能力，但命令执行与整机触摸注入这两个原语改为 fail-closed 知情同意。判据是「一次调用是否触发多次模型调用且不可取消」，而非主观的危险度——这条判据让黑白名单不会随功能增加而失控膨胀。

04

全量自持执行层与感知层：不让能力外挂变成用户的前置负担

proot 自持 Ubuntu 跑在 App 进程内，随包仅 214 KB，rootfs 与组件按需下载；无障碍自持提供零弹窗截屏读屏。这两个方案都刻意不依赖外部 App——Termux 方案能跑但拿不到 stdout、跨 App 保活做不到、无障碍授权只用户手动开一次。

05

结构化省 token：用装配策略而非描述精简对付上下文成本

认识到成本来自「每轮全量下发 Schema」这个结构，因此用闲聊快路与意图路由从装配层解决。更关键的是明确覆盖率不是 KPI，误放率才是——误放会让模型在残缺工具集里自信作答，比漏放更难发现。

06

构建期门禁替代托管 CI：把质量约束落在跑不掉的钩子上

四道门禁挂在 preBuild（协议双端同源、双语 key 对齐、关键链路空 catch 零容忍、全仓空 catch 棘轮预算只降不升），两道发布闸门挂在 packageRelease（工作区必须干净、单测必须全绿）。本地每次构建即触发，不依赖任何外部服务。

工程上几个值得单独说的点

4.2

— **协议双端同源且构建期强制。**手机端与眼镜端的协议常量逐字节一致，任一端改动未同步，另一端构建即失败。这消灭了一整类「改了 A 忘了改 B」的线上故障。

— **单色屏是一等设计约束。**眼镜是单绿色波导，深蓝灰在此屏幕上等同于黑色。因此 UI 生成侧有独立的配色规范约束——这不是审美选择，是可见性问题。

— **测试覆盖历史事故而非最好测的代码。**844 个用例大量落在「真出过 bug 的高危文件」上：ADB sync 协议的 FAIL 分支、HID 描述符字节矩阵、审批闸门八道 guard 的合成语义、聊天历史崩溃截断后的恢复。

— **空 catch 被当成决策来治理。**关键链路要求零容忍，其余位置有只降不升的棘轮预算，且必须标注原因——把「默默吞掉」变成「显式声明的决策」。

— **大改动的安全性靠结构而非纪律。**god class 拆分后保留全部 public 方法作委派 facade，调用方零改动；会话记忆从整表重写改为 JSONL 追加后，崩溃留下的半行会被跳过而不是毁掉全部历史。

05

工程数据

规模、质量门禁与版本演进

规模指标 5.1

全部由仓库实测得出

105 AI 工具（静态注册）	**21** ToolProvider（按域拆分）	**17** 工具域
844 单元测试用例	**345** Kotlin 源文件	**113** AI 能力层文件

国际化与适配 5.2

项	口径	保障机制
语言条目	手机端中文 1774 / 英文 1774；眼镜端 24	checkI18nKeysSynced 校验两模块 zh□en key 集合相等，不一致直接构建失败
运行时切换	AppCompatDelegate 设置应用语言，无需重启	首次启动自动检测系统语言；Crowdin 云端协作翻译
国内 ROM	华为 EMUI 12+ / 小米 MIUI 14+ / OPPO ColorOS 13+ / vivo Funtouch OS 13+	投屏黑屏自动修复（HWC 禁用 + 软件渲染 + 分辨率降级）、电池优化白名单与自启动引导、悬浮窗权限检测
芯片与栈	联发科 / 麒麟；QTI 蓝牙栈	编码器回退与分辨率自动降级；HID sendReport 自动降级与 HOGP 手动开启引导
包体控制	眼镜端 R8 混淆后约 8.7 MB（优化前 13.8 MB）	OCR 模型改为按需下载，随包仅保留 proot 两个 so 共 214 KB

质量门禁 5.3

本地每次构建即触发，不依赖外部服务

门禁	挂载点	拦住什么
checkProtocolSynced	preBuild	双端协议常量不同源；业务文件出现裸协议字面量
checkI18nKeysSynced	preBuild	两模块中英 key 集合不一致
checkKeyPathEmptyCatch	preBuild	关键链路文件出现未标注的空 catch（6 个文件零容忍）
checkNoBareCatch	preBuild	全仓空 catch 超出棘轮预算，只降不升
check_tool_wiring.py	改工具后手动跑	Provider 名单与声明表不一致；声明缺风险档 / schema / 文案
checkGitClean	packageRelease	工作区非干净即出包失败（阻止「发布的不是仓库里的东西」）
testDebugUnitTest	packageRelease 依赖	单测没绿出不了包

「不建托管 CI」是一个明确的结论而非省事：四道门禁已挂在 preBuild，本地每次构建都会跑；Android CI 需要复刻 SDK / NDK / 16KB

校验环境，而本项目是单作者、唯一发布路径就是本地构建。真正需要强制的两条——出包必跑测试、脏工作区不许出包——用 Gradle 闸门闭合更可靠。

版本演进（近期主线） 5.4

版本	主线变化
v4.0	会话架构接缝化（能力可插拔 + 全程可回放）；IMU 头动规则；外部 MCP 客户端与 OAuth；知识库混合检索；引导与权限收敛；签名材料出库（缺密钥即中止 release 构建）
v4.1	屏幕操作域（无障碍）5 工具；本机网页预览；12 家品牌模型供应商预设与密钥钥匙串加密；聊天富卡片全离线渲染；ASR 推送改 WiFi 优先 + 蓝牙兜底双通道
v4.2	主动式陪伴（5 场景 × 4 频率 × 7 项主动能力）；环境聆听两级 VAD；MCP 本地 stdio 桥；长期记忆混合检索 RRF 融合
v4.3	结构性省 token 两刀（闲聊快路 + 工具意图路由）；聆听模式持续对话重构（句子聚合 / 回答期真闭麦 / 空闲常开）
v4.4	长期记忆 mem0 式评分淘汰；Token 精确计量；子代理双型（只读 / 可写各占独立槽位）；MCP OAuth PKCE + 动态客户端注册；四个新工具；云端 TTS 音色管理；会话文件上限 50

下一步 把远端推理引擎收敛为单一 MCP server，手机端与眼镜端只保留「入口 + 执行 + 感知」，编译能力不下沉到设备侧；同时补上显式反射测试阶段，把目前唯一缺失的安全验证环节补齐。